Lec 05 Monte Carlo Learning

The simplest MC-based RL algorithm
- Description of the algorithm
- **Pseudocode: MC Basic algorithm (a model-free variant of policy iteration)**

Use data more efficiently
- Visit
- When to update the policy

MC Exploring Starts
- **Pseudocode: MC Exploring algorithm (a sample-effcient variant of policy iteration)**

MC without exploring starts
- Soft policies
  - What is an ϵ- greedy policy?
  - Why use ϵ-greedy? Balance between exploitation and exploration
- How to embed ϵ-greedy into the MC-based RL algorithms?
- **Pseudocode: MC ϵ-greedy (a variant of MC Exploring Starts)**
- Advantages and Disadvantages of ϵ greedy Policies

Value iteration and policy iteration: model-based reinforcement learning(MBRL)

This lecture introduces

The simplest MC-based RL algorithm

- Starting from (s, a), following policy π_k, generate an episode.
- The return of this episode is $g(s, a)$
- $g(s, a)$ is a sample of G_t in

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- Suppose we have a set of episodes and hence $\{g^{(j)}(s, a)\}$.
  Then,

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^{N} g^{(i)}(s, a)$$

Fundamental idea: When model is unavailable, we can use data.

Description of the algorithm

▷ Description of the algorithm:

Given an initial policy π_0, there are two steps at the kth iteration.

- **Step 1: policy evaluation.** This step is to obtain $q_{\pi_k}(s,a)$ for all (s,a). Specifically, for each action-state pair (s,a), run an infinite number of (or sufficiently many) episodes. The average of their returns is used to approximate $q_{\pi_k}(s,a)$.
- **Step 2: policy improvement.** This step is to solve $\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a|s) q_{\pi_k}(s,a)$ for all $s \in \mathcal{S}$. The greedy optimal policy is $\pi_{k+1}(a_k^*|s) = 1$ where $a_k^* = \arg\max_a q_{\pi_k}(s,a)$.

Exactly the same as the policy iteration algorithm, except

- Estimate $q_{\pi_k}(s,a)$ directly, instead of solving $v_{\pi_k}(s)$.

> In fact, the cores of policy iteration and MC basic are same, the only difference lies in the first step:
>
> - Policy iteration: first compute the state value, then compute action value(q_{π_k})
> - MC Basics: Estimate $q_{\pi_k}(s,a)$ directly by using data

Pseudocode: MC Basic algorithm (a model-free variant of policy iteration)

Initialization: Initial guess π_0.

Aim: Search for an optimal policy.

While the value estimate has not converged, for the kth iteration, do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

Collect sufficiently many episodes starting from (s, a) following π_k

MC-based policy evaluation step:

$q_{\pi_k}(s, a) =$ average return of all the episodes starting from (s, a)

Policy improvement step:

$a_k^*(s) = \arg\max_a q_{\pi_k}(s, a)$

$\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

> **Episode length:**
>
> - When the episode length is short, only the states that are close to the target have nonzero state values.
> - As the episode length increases, the states closer to the target will have nonzero values earlier than those farther away.
> - The episode length should be sufficiently long, but does not have to be infinitely long.

Use data more efficiently

The MC Basic algorithm:

- **Advantage:** reveal the core idea clearly
- **Disadvantage:** too simple to be practical

Visit

Consider a grid-world example, following a policy π, we can get an episode such as:

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$$

🏷️ **Definition:**
Visit: every time a state-action pair appears in the episode, it is called a visit of the state-action pair.

Methods to approximate $q_{\pi_k}(s, a)$ using the data:

- **Initial-visit method:** Just calculate the return and approximate $q_{\pi_k}(s, a)$
  - This is what the MC Basic algorithm does. But it does not fully utilize the data, because in the episode above, pairs like (s_2, a_2) are also visited so that we can use them to update other visit.
- **Data-efficient methods:**

$$
\begin{aligned}
s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \ldots & [\text{original episode}] \\
s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \ldots & [\text{episode starting from}(s_2, a_4)] \\
s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \ldots & [\text{episode starting from}(s_1, a_2)] \\
s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \ldots & [\text{episode starting from}(s_2, a_3)] \\
s_5 \xrightarrow{a_1} \ldots & [\text{episode starting from}(s_5, a_1)]
\end{aligned}
$$

  Can estimate $q_\pi(s_1, a_2), q_\pi(s_2, a_4), q_\pi(s_2, a_3), q_\pi(s_5, a_1), \ldots$
  - first-visit method
  - every-visit method

When to update the policy

- **Low efficiency:** In the policy evaluation step, to collect all the episodes starting from a
  state-action pair and then use the average return to approximate the action value.
- **High efficiency:** Uses the return of a single episode to approximate the action value. In
  this way, we can improve the policy episode-by-episode.
- **Will the second method cause problems?**
  - In fact, we have applied its idea in the truncated policy iteration algorithm (i.e.

approximating v_{π_k} via limited iterations).

- Even if using the single episode we sampled as $q(s, a)$ is imprecise, as long as the relative values of $q(s, a)$ for different a are roughly correct, the policy still selects the
action with higher return in $\arg\max_a q_k(s, a)$, thus improving on the right track.
- When the policy gets better, it generates episodes that are closer to the optimal trajectory, and consequently less random (i.e. the variance of $q_k(s, a)$ gets smaller and thus $q_k(s, a)$ converges to $q_{\pi_k}(s, a)$ continuously).

MC Exploring Starts

Pseudocode: MC Exploring algorithm (a sample-effcient variant of policy iteration)

Initialization: Initial guess π_0.
Aim: Search for an optimal policy.

For each episode, do
  Episode generation: Randomly select a starting state-action pair (s_0, a_0) and ensure that all pairs can be possibly selected. Following the current policy, generate an episode of length T: $s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$.
  Policy evaluation and policy improvement:
  Initialization: $g \leftarrow 0$
  For each step of the episode, $t = T-1, T-2, \ldots, 0$, do
    $g \leftarrow \gamma g + r_{t+1}$
    Use the first-visit strategy:
    If (s_t, a_t) does not appear in $(s_0, a_0, s_1, a_1, \ldots, s_{t-1}, a_{t-1})$, then
      $Returns(s_t, a_t) \leftarrow Returns(s_t, a_t) + g$
      $q(s_t, a_t) = \text{average}(Returns(s_t, a_t))$
      $\pi(a|s_t) = 1$ if $a = \arg\max_a q(s_t, a)$

- Here `For each step of the episode, t = T - 1, T - 2, ..., 0, do` is a trick to improve compute efficiency:
Consider the following episode: $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \ldots$

We firstly calculate $q(s_5, a_1)$, then get $q(s_2, a_3)$ by adding r_{t+1} and $q(s_5, a_1)$. Others can be compute in the same way. It will be more compute efficient.

What is exploring starts? Exploring starts means we need to generate sufficiently many episodes $\underbrace{\textit{starting}}_{\textit{starts}}$ *from* $\underbrace{\textit{every}}_{\textit{exploring}}$ *state-action pair.*

- **Why do we need to consider exploring starts?**
  - In theory, only if every action value for every state is well explored, can we select the optimal actions correctly.
  - On the contrary, if an action is not explored, this action may happen to be the optimal one and hence be missed.

MC without exploring starts

Soft policies

- A policy is called *soft* if the probability to take any action is positive.
- Why introduce soft policies?
  - With a soft policy, a few episodes that are sufficiently long can visit every state-action pair for sufficiently many times.
  - Then, we do not need to have a large number of episodes starting from every state-action pair. Hence, the requirement of exploring starts can thus be removed.

What is an ϵ- greedy policy?

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1), & \text{for the greedy action,} \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{for the other } |\mathcal{A}(s)| - 1 \text{ actions.} \end{cases}$$

where $\epsilon \in [0, 1]$ and $|\mathcal{A}(s)|$ is the number of actions for s.

- The chance to choose the greedy action is always greater than other actions, because $1 - \frac{\epsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) = 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} \geq \frac{\epsilon}{|\mathcal{A}(s)|}$.

Why use ϵ-greedy? Balance between exploitation and exploration

- When $\epsilon = 0$, it becomes greedy! Less exploration but more exploitation!
- When $\epsilon = 1$, it becomes a uniform distribution. More exploration but less exploitation.

How to embed ϵ-greedy into the MC-based RL algorithms?

Now, the policy improvement step is changed to solve

$$\pi_{k+1}(s) = \arg\max_{\pi \in \Pi_\epsilon} \sum_a \pi(a|s) q_{\pi_k}(s,a),$$

where Π_ϵ denotes the set of all ϵ-greedy policies with a fixed value of ϵ.
The optimal policy here is

$$\pi_{k+1}(a|s) = \begin{cases} 1 - \frac{|\mathcal{A}(s)|-1}{|\mathcal{A}(s)|}\epsilon, & a = a_k^{,} \\ \frac{1}{|\mathcal{A}(s)|}\epsilon, & a \neq a_k^{,} \end{cases}$$

- **MC ϵ-Greedy** is the *same* as that of **MC Exploring Starts** *except* that the former uses ϵ-greedy policies.
- **It does not require exploring starts**, but still requires to visit all state-action pairs in a different form.

Pseudocode: MC ϵ-greedy (a variant of MC Exploring Starts)

Pseudocode: MC ϵ-Greedy (a variant of MC Exploring Starts)

Initialization: Initial guess π_0 and the value of $\epsilon \in [0, 1]$
Aim: Search for an optimal policy.

For each episode, do

Episode generation: Randomly select a starting state-action pair (s_0, a_0). Following the current policy, generate an episode of length T: $s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$.

Policy evaluation and policy improvement:

Initialization: $g \leftarrow 0$

For each step of the episode, $t = T-1, T-2, \ldots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

Use the every-visit method:

If (s_t, a_t) does not appear in $(s_0, a_0, s_1, a_1, \ldots, s_{t-1}, a_{t-1})$, then

$Returns(s_t, a_t) \leftarrow Returns(s_t, a_t) + g$

$q(s_t, a_t) = \text{average}(Returns(s_t, a_t))$

Let $a^* = \arg\max_a q(s_t, a)$ and

$$\pi(a|s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)|-1}{|\mathcal{A}(s_t)|}\epsilon, & a = a^* \\ \frac{1}{|\mathcal{A}(s_t)|}\epsilon, & a \neq a^* \end{cases}$$

> **Attention:** `If (s_t, a_t) does not appear in (s_0, a_0, s_1, a_1, \dots, s_{t-1}, a_{t-1})` should be removed, as here every-visit method is used.

Advantages and Disadvantages of ϵ greedy Policies

- **Advantage:** They have strong exploration ability whenϵ is large.
- **Disadvantage:** They are not optimal in general.
  - ϵ cannot be too large.
  - A common technique is to use a **decaying** ϵ (reducing ϵ over time as the agent learns).